

Code Appendix README

Classifying Pre-Labeled Robot Wall-Following Commands from Ultrasonic Sensor Data

Purpose

This appendix documents the code used to run the machine learning analysis. The project uses Python and scikit-learn to load the public UCI Wall-Following Robot Navigation Data Set, train classification models, select hyperparameters with cross-validation, evaluate held-out test performance, and generate the plots/tables used in the report.

How to Reproduce

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python main.py
```

Wrapper Code

`main.py` is the wrapper script. It calls the supporting modules in `src/` to:

1. Create required folders.
2. Download or locate the UCI raw data files.
3. Load the 24-sensor dataset and assign feature names `US_1` through `US_24`.
4. Print dataset shape, feature count, class labels, class counts, missing values, and split sizes.
5. Create a fixed stratified 80/20 train/test split.
6. Generate EDA outputs: class distribution, PCA plot, sensor statistics, and correlation heatmap.
7. Train models with scikit-learn Pipeline and GridSearchCV.
8. Evaluate each model on training data and the held-out test set.
9. Save metrics, best parameters, classification reports, confusion matrices, figures, and trained models.
10. Run the optional reduced-sensor ablation study for the portfolio extension.

The wrapper does not generate the final report text. The report is written from the saved results produced by `python main.py`.

Source File Map

File	Purpose
<code>main.py</code>	Wrapper script that runs the full experiment end to end.
<code>src/config.py</code>	Stores paths, random seed, dataset URLs, feature names, labels, and model settings.
<code>src/download_data.py</code>	Downloads or locates the UCI raw dataset files.
<code>src/load_data.py</code>	Parses raw <code>.data</code> files, assigns feature/target names, checks labels, and saves processed CSVs.
<code>src/train_models.py</code>	Defines scikit-learn pipelines, model grids, StratifiedKFold, and GridSearchCV.
<code>src/evaluate_models.py</code>	Computes accuracy, macro precision, macro recall, macro F1, classification reports, and confusion matrices.
<code>src/plot_results.py</code>	Generates class distribution, PCA, model comparison, CV, confusion matrix, ablation, and coefficient figures.
<code>src/utils.py</code>	Provides helper functions for directories, JSON/text output, filename slugs, and metric formatting.

Models and Model Selection

The class-report comparison uses Dummy Majority, Logistic Regression, Linear SVM, and RBF Kernel SVM. GridSearchCV with stratified 5-fold cross-validation selects model parameters using macro F1.

Model	Parameters selected with cross-validation
Logistic Regression	C
Linear SVM	C
RBF SVM	C, gamma

Preprocessing is inside each scikit-learn Pipeline, so scaling is fit only on training folds during cross-validation.

Verification Target

This is offline supervised classification. The code verifies a prediction by comparing:

```
predicted_command == true_command
```

The project does not estimate robot position, measure physical standoff error, deploy a robot controller, or use reinforcement learning.

Main Generated Outputs

Output	Purpose
results/dataset_verification.csv	Dataset source, file name, feature count, class counts, missing values, split sizes, and verification target.
results/model_metrics.csv	Train/test accuracy, macro precision, macro recall, and macro F1 for all full models.
results/class_model_metrics.csv	Class-report model comparison: Dummy, Logistic Regression, Linear SVM, and RBF SVM.
results/cv_results.csv	Cross-validation scores for each tested hyperparameter setting.
results/best_params.json	Best hyperparameters selected by GridSearchCV.
classification_reports/*.txt	Per-class precision, recall, and F1 reports saved under results/.
confusion_matrices/*.csv	Held-out test confusion matrices saved under results/.
figures/*.png	Plots used in the report and poster.

Full Code Listing

The full source listing below includes the wrapper code and supporting modules used to generate the project outputs.

main.py

```
1 from __future__ import annotations
2
3 import json
4 import os
5 import shutil
6 import warnings
7
8 os.environ.setdefault("PYTHONWARNINGS", "ignore::FutureWarning")
9 warnings.filterwarnings("ignore", category=FutureWarning)
10
11 import joblib
12 import pandas as pd
13 from sklearn.model_selection import train_test_split
14
15 from src.config import (
16     CLASSIFICATION_REPORT_DIR,
17     COMMAND_ORDER,
18     CONFUSION_MATRIX_DIR,
19     DATASET_FILES,
20     DATASET_URLS,
21     FIGURES_DIR,
22     PRIMARY_SCORING,
23     PROJECT_ROOT,
24     RANDOM_STATE,
25     RESULTS_DIR,
26     TEST_SIZE,
27     TRAINED_MODEL_DIR,
28 )
29 from src.download_data import download_all_datasets
30 from src.evaluate_models import (
31     evaluate_estimator,
32     extract_linear_feature_importance,
33     most_confused_pair,
34 )
35 from src.load_data import RobotDataset, load_all_datasets, summarize_missing_values
36 from src.plot_results import (
37     save_class_distribution,
38     save_coefficient_plot,
39     save_confusion_matrix,
40     save_cv_score_comparison,
41     save_metric_comparison,
42     save_pca_sensor_space,
43     save_sensor_ablation_comparison,
44     save_sensor_correlation_heatmap,
45 )
46 from src.train_models import (
47     cv_results_to_frame,
48     fit_grid_search,
49     get_ablation_model_specs,
50     get_main_model_specs,
51 )
52 from src.utils import (
53     ensure_project_directories,
54     format_metric,
55     slugify,
56     write_json,
57     write_text,
58 )
59
60 CLASS_REPORT_MODELS = [
61     "Dummy Majority",
62     "Logistic Regression",
63     "Linear SVM",
64     "RBF SVM",
65 ]
66 CLASS_SELECTION_MODELS = ["Logistic Regression", "Linear SVM", "RBF SVM"]
67
68
69 def print_dataset_summary(dataset: RobotDataset) -> None:
70     """Print the raw data facts that matter for grading and reproducibility."""
71     missing_counts = summarize_missing_values(dataset)
72     print("\nDataset summary")
73     print("=====")
74     print(f"Dataset: {dataset.display_name}")
75     print(f"Raw file used: {DATASET_FILES[dataset.key]}")
76     print(f"Shape: {dataset.data.shape[0]} samples x {dataset.data.shape[1]} columns")
77     print(f"Feature count: {len(dataset.feature_names)}")
78     print("Class counts:")
79     print(dataset.y.value_counts().reindex(COMMAND_ORDER).to_string())
80     print(f"Label names: {' '.join(COMMAND_ORDER)}")
81     print(f"Missing values: {int(missing_counts.sum())}")
82
83
84 def make_train_test_indices(y: pd.Series) -> tuple[pd.Index, pd.Index]:
85     """Create one stratified train/test split reused across all experiments."""
86     indices = pd.Index(range(len(y)))
```

```

87     train_idx, test_idx = train_test_split(
88         indices,
89         test_size=TEST_SIZE,
90         stratify=y,
91         random_state=RANDOM_STATE,
92     )
93     return pd.Index(train_idx), pd.Index(test_idx)
94
95
96 def print_split_summary(dataset: RobotDataset, train_idx: pd.Index, test_idx: pd.Index) -> None:
97     """Show that the held-out split preserved the class imbalance."""
98     print("\nTrain/test split")
99     print("=====")
100    print(f"Train set size: {len(train_idx)}")
101    print(f"Test set size: {len(test_idx)}")
102    print("Train class counts:")
103    print(dataset.y.iloc[train_idx].value_counts().reindex(COMMAND_ORDER).to_string())
104    print("Test class counts:")
105    print(dataset.y.iloc[test_idx].value_counts().reindex(COMMAND_ORDER).to_string())
106
107
108 def write_dataset_verification(
109     dataset: RobotDataset, train_idx: pd.Index, test_idx: pd.Index
110 ) -> pd.DataFrame:
111     """Save a compact audit table describing the dataset and verification target."""
112     full_counts = dataset.y.value_counts().reindex(COMMAND_ORDER).fillna(0).astype(int)
113     train_counts = dataset.y.iloc[train_idx].value_counts().reindex(COMMAND_ORDER).fillna(0).astype(int)
114     test_counts = dataset.y.iloc[test_idx].value_counts().reindex(COMMAND_ORDER).fillna(0).astype(int)
115     full_props = full_counts / len(dataset.y)
116     train_props = train_counts / len(train_idx)
117     test_props = test_counts / len(test_idx)
118     max_train_diff = float((train_props - full_props).abs().max())
119     max_test_diff = float((test_props - full_props).abs().max())
120     stratification_check = (
121         "pass: stratified split preserved class proportions within "
122         f"{max(max_train_diff, max_test_diff):.4f} absolute proportion"
123     )
124
125     rows = [
126         ("dataset_source", "UCI Wall-Following Robot Navigation Data Set"),
127         ("dataset_url", DATASET_URLS[dataset.key]),
128         ("raw_file_name", DATASET_FILES[dataset.key]),
129         ("number_of_samples", len(dataset.data)),
130         ("number_of_features", len(dataset.feature_names)),
131         ("feature_names", ", ".join(dataset.feature_names)),
132         ("target_column_name", dataset.target_name),
133         ("unique_command_labels", ", ".join(COMMAND_ORDER)),
134         ("class_counts", json.dumps(full_counts.to_dict(), sort_keys=True)),
135         ("missing_value_count", int(dataset.data.isna().sum().sum())),
136         ("train_set_size", len(train_idx)),
137         ("test_set_size", len(test_idx)),
138         ("train_class_counts", json.dumps(train_counts.to_dict(), sort_keys=True)),
139         ("test_class_counts", json.dumps(test_counts.to_dict(), sort_keys=True)),
140         ("stratification_check", stratification_check),
141         ("random_seed", RANDOM_STATE),
142         (
143             "preprocessing_statement",
144             "StandardScaler preprocessing is fit inside scikit-learn Pipeline during GridSearchCV.",
145         ),
146         (
147             "verification_target",
148             "predicted command label compared with dataset-provided true command label",
149         ),
150     ]
151     verification = pd.DataFrame(rows, columns=["field", "value"])
152     verification.to_csv(RESULTS_DIR / "dataset_verification.csv", index=False)
153     return verification
154
155
156 def run_main_experiment(
157     dataset: RobotDataset, train_idx: pd.Index, test_idx: pd.Index
158 ) -> tuple[pd.DataFrame, dict, dict[str, pd.DataFrame], pd.DataFrame]:
159     """Train and evaluate the full 24-sensor model comparison."""
160     X_train = dataset.X.iloc[train_idx]
161     X_test = dataset.X.iloc[test_idx]
162     y_train = dataset.y.iloc[train_idx]
163     y_test = dataset.y.iloc[test_idx]
164
165     metric_rows = []
166     cv_frames = []
167     best_params = {}
168     confusion_matrices = {}
169     feature_importance_frames = []
170
171     print("\nTraining full 24-sensor models")
172     print("=====")
173     for spec in get_main_model_specs():
174         print(f"Fitting {spec.name} with GridSearchCV ({PRIMARY_SCORING})")
175         search = fit_grid_search(spec, X_train, y_train)

```

```

176     estimator = search.best_estimator_
177     model_slug = slugify(spec.name)
178
179     metrics, report_text, cm_frame = evaluate_estimator(
180         spec.name, estimator, X_train, y_train, X_test, y_test
181     )
182     metrics["dataset"] = dataset.key
183     metrics["feature_set"] = dataset.display_name
184     metrics["n_features"] = len(dataset.feature_names)
185     metrics["cv_best_macro_f1"] = float(search.best_score_)
186     metrics["best_params"] = json.dumps(search.best_params_, sort_keys=True)
187     metric_rows.append(metrics)
188
189     best_params[spec.name] = {
190         "best_params": search.best_params_,
191         "best_cv_macro_f1": float(search.best_score_),
192     }
193     cv_frames.append(cv_results_to_frame(spec.name, search, dataset.key))
194
195     write_text(CLASSIFICATION_REPORT_DIR / f"{model_slug}.txt", report_text)
196     cm_frame.to_csv(CONFUSION_MATRIX_DIR / f"{model_slug}.csv")
197     save_confusion_matrix(
198         cm_frame,
199         FIGURES_DIR / f"confusion_matrix_{model_slug}.png",
200         f"Held-out Test Confusion Matrix: {spec.name}",
201     )
202     confusion_matrices[spec.name] = cm_frame
203     joblib.dump(estimator, TRAINED_MODEL_DIR / f"{model_slug}.joblib")
204
205     if spec.name in {"Logistic Regression", "Linear SVM"}:
206         feature_importance_frames.append(
207             extract_linear_feature_importance(
208                 spec.name, estimator, dataset.feature_names
209             )
210         )
211
212     metrics_frame = pd.DataFrame(metric_rows)
213     best_model_name = metrics_frame.sort_values(
214         ["test_f1_macro", "test_accuracy"], ascending=False
215     ).iloc[0]["model"]
216     metrics_frame["is_best_model"] = metrics_frame["model"].eq(best_model_name)
217     metrics_frame = metrics_frame.sort_values("test_f1_macro", ascending=False)
218     metrics_frame.to_csv(RESULTS_DIR / "model_metrics.csv", index=False)
219
220     class_metrics = metrics_frame[metrics_frame["model"].isin(CLASS_REPORT_MODELS)].copy()
221     class_best_model_name = (
222         class_metrics[class_metrics["model"].isin(CLASS_SELECTION_MODELS)]
223         .sort_values(["test_f1_macro", "test_accuracy"], ascending=False)
224         .iloc[0]["model"]
225     )
226     class_metrics["is_class_report_best_model"] = class_metrics["model"].eq(class_best_model_name)
227     class_metrics = class_metrics.sort_values("test_f1_macro", ascending=False)
228     class_metrics.to_csv(RESULTS_DIR / "class_model_metrics.csv", index=False)
229
230     cv_results = pd.concat(cv_frames, ignore_index=True)
231     cv_results.to_csv(RESULTS_DIR / "cv_results.csv", index=False)
232
233     feature_importance = (
234         pd.concat(feature_importance_frames, ignore_index=True)
235         if feature_importance_frames
236         else pd.DataFrame()
237     )
238     if not feature_importance.empty:
239         feature_importance.to_csv(RESULTS_DIR / "feature_importance.csv", index=False)
240         for model_name in ["Logistic Regression", "Linear SVM"]:
241             save_coefficient_plot(
242                 feature_importance,
243                 model_name,
244                 FIGURES_DIR / f"sensor_coefficients_{slugify(model_name)}.png",
245             )
246
247     save_metric_comparison(
248         metrics_frame,
249         "f1_macro",
250         FIGURES_DIR / "model_comparison_f1_all.png",
251         "Full Portfolio Model Comparison: Macro F1",
252         "Macro F1",
253     )
254     save_metric_comparison(
255         metrics_frame,
256         "accuracy",
257         FIGURES_DIR / "model_comparison_accuracy_all.png",
258         "Full Portfolio Model Comparison: Accuracy",
259         "Accuracy",
260     )
261     save_metric_comparison(
262         class_metrics,
263         "f1_macro",
264         FIGURES_DIR / "model_comparison_f1.png",

```

```

265         "Class Report Model Comparison: Macro F1",
266         "Macro F1",
267     )
268     save_metric_comparison(
269         class_metrics,
270         "accuracy",
271         FIGURES_DIR / "model_comparison_accuracy.png",
272         "Class Report Model Comparison: Accuracy",
273         "Accuracy",
274     )
275     save_cv_score_comparison(metrics_frame, FIGURES_DIR / "cv_score_comparison.png")
276     shutil.copyfile(
277         FIGURES_DIR / f"confusion_matrix_{slugify(best_model_name)}.png",
278         FIGURES_DIR / "confusion_matrix_best_overall_model.png",
279     )
280     shutil.copyfile(
281         FIGURES_DIR / f"confusion_matrix_{slugify(class_best_model_name)}.png",
282         FIGURES_DIR / "confusion_matrix_best_model.png",
283     )
284
285     return metrics_frame, best_params, confusion_matrices, feature_importance
286
287
288 def run_sensor_ablation(
289     datasets: dict[str, RobotDataset],
290     train_idx: pd.Index,
291     test_idx: pd.Index,
292 ) -> tuple[pd.DataFrame, dict]:
293     """Compare the full 24-sensor file with official reduced-sensor UCI files."""
294     rows = []
295     best_params = {}
296     print("\nRunning sensor ablation study")
297     print("=====")
298     for dataset_key in ["24_sensors", "4_sensors", "2_sensors"]:
299         dataset = datasets[dataset_key]
300         X_train = dataset.X.iloc[train_idx]
301         X_test = dataset.X.iloc[test_idx]
302         y_train = dataset.y.iloc[train_idx]
303         y_test = dataset.y.iloc[test_idx]
304         best_params[dataset_key] = {}
305         for spec in get_ablation_model_specs():
306             print(f"Ablation: {dataset.display_name} | {spec.name}")
307             search = fit_grid_search(spec, X_train, y_train)
308             metrics, _, _ = evaluate_estimator(
309                 spec.name, search.best_estimator_, X_train, y_train, X_test, y_test
310             )
311             metrics.update(
312                 {
313                     "dataset": dataset_key,
314                     "feature_set": dataset.display_name,
315                     "n_features": len(dataset.feature_names),
316                     "cv_best_macro_f1": float(search.best_score_),
317                     "best_params": json.dumps(search.best_params_, sort_keys=True),
318                     "source_note": dataset.source_note,
319                 }
320             )
321             rows.append(metrics)
322             best_params[dataset_key][spec.name] = {
323                 "best_params": search.best_params_,
324                 "best_cv_macro_f1": float(search.best_score_),
325             }
326
327     ablation = pd.DataFrame(rows).sort_values(
328         ["feature_set", "test_f1_macro"], ascending=[True, False]
329     )
330     ablation.to_csv(RESULTS_DIR / "sensor_ablation_results.csv", index=False)
331     save_sensor_ablation_comparison(
332         ablation, FIGURES_DIR / "sensor_ablation_comparison.png"
333     )
334     return ablation, best_params
335
336
337 def generate_eda_outputs(dataset: RobotDataset) -> pd.DataFrame:
338     """Create lightweight exploratory tables and figures for the report."""
339     print("\nGenerating EDA outputs")
340     print("=====")
341     summary = dataset.X.agg(["mean", "std", "min", "max"]).T
342     summary.index.name = "sensor"
343     summary.to_csv(RESULTS_DIR / "sensor_summary_statistics.csv")
344     save_class_distribution(dataset.y, FIGURES_DIR / "class_distribution.png")
345     save_pca_sensor_space(dataset.X, dataset.y, FIGURES_DIR / "pca_sensor_space.png")
346     save_sensor_correlation_heatmap(
347         dataset.X, FIGURES_DIR / "sensor_correlation_heatmap.png"
348     )
349     return summary
350
351
352 def build_context(
353     primary: RobotDataset,

```

```

354 metrics: pd.DataFrame,
355 best_params: dict,
356 ablation: pd.DataFrame,
357 feature_importance: pd.DataFrame,
358 confusion_matrices: dict[str, pd.DataFrame],
359 ) -> dict[str, object]:
360     """Collect headline results used only for terminal summary output."""
361     best_row = metrics.sort_values(["test_f1_macro", "test_accuracy"], ascending=False).iloc[0]
362     best_model_name = str(best_row["model"])
363     confusion = most_confused_pair(confusion_matrices[best_model_name])
364     class_metrics = metrics[metrics["model"].isin(CLASS_REPORT_MODELS)].copy()
365     class_best_row = (
366         class_metrics[class_metrics["model"].isin(CLASS_SELECTION_MODELS)]
367         .sort_values(["test_f1_macro", "test_accuracy"], ascending=False)
368         .iloc[0]
369     )
370     class_best_model_name = str(class_best_row["model"])
371     class_confusion = most_confused_pair(confusion_matrices[class_best_model_name])
372
373     nonlinear_models = ["RBF SVM", "MLP Neural Network"]
374     linear_models = ["Logistic Regression", "Linear SVM"]
375     best_nonlinear = metrics[metrics["model"].isin(nonlinear_models)].sort_values(
376         "test_f1_macro", ascending=False
377     ).iloc[0]
378     best_linear = metrics[metrics["model"].isin(linear_models)].sort_values(
379         "test_f1_macro", ascending=False
380     ).iloc[0]
381     nonlinear_beats_linear = bool(best_nonlinear["test_f1_macro"] > best_linear["test_f1_macro"])
382
383     rbf_row = metrics[metrics["model"] == "RBF SVM"].iloc[0]
384     mlp_row = metrics[metrics["model"] == "MLP Neural Network"].iloc[0]
385
386     ablation_best = ablation.sort_values(
387         ["dataset", "test_f1_macro"], ascending=[True, False]
388     ).groupby("dataset", as_index=False).head(1)
389     full_ablation_f1 = float(
390         ablation_best.loc[ablation_best["dataset"] == "24_sensors", "test_f1_macro"].iloc[0]
391     )
392     reduced_best_f1 = float(
393         ablation_best.loc[
394             ablation_best["dataset"].isin(["4_sensors", "2_sensors"]), "test_f1_macro"
395         ].max()
396     )
397     sensor_reduction_hurt = full_ablation_f1 > reduced_best_f1
398
399     top_features = pd.DataFrame()
400     if not feature_importance.empty:
401         top_features = feature_importance.groupby("model", as_index=False).head(5)
402
403     class_counts = primary.y.value_counts().reindex(COMMAND_ORDER)
404
405     return {
406         "best_row": best_row,
407         "best_model_name": best_model_name,
408         "best_params": best_params["full_24_sensor_experiment"][best_model_name]["best_params"],
409         "confusion": confusion,
410         "class_best_row": class_best_row,
411         "class_best_model_name": class_best_model_name,
412         "class_best_params": best_params["full_24_sensor_experiment"][class_best_model_name]["best_params"],
413         "class_confusion": class_confusion,
414         "nonlinear_beats_linear": nonlinear_beats_linear,
415         "best_nonlinear_model": str(best_nonlinear["model"]),
416         "best_nonlinear_f1": float(best_nonlinear["test_f1_macro"]),
417         "best_linear_model": str(best_linear["model"]),
418         "best_linear_f1": float(best_linear["test_f1_macro"]),
419         "rbf_test_f1": float(rbf_row["test_f1_macro"]),
420         "mlp_test_f1": float(mlp_row["test_f1_macro"]),
421         "sensor_reduction_hurt": sensor_reduction_hurt,
422         "full_ablation_f1": full_ablation_f1,
423         "reduced_best_f1": reduced_best_f1,
424         "ablation_best": ablation_best,
425         "top_features": top_features,
426         "class_counts": class_counts,
427         "missing_values": int(primary.data.isna().sum().sum()),
428     }
429
430
431 def verify_required_outputs() -> list[str]:
432     """Verify only analysis artifacts generated by this script."""
433     model_slugs = [slugify(spec.name) for spec in get_main_model_specs()]
434     required_paths = [
435         RESULTS_DIR / "dataset_verification.csv",
436         RESULTS_DIR / "class_model_metrics.csv",
437         RESULTS_DIR / "model_metrics.csv",
438         RESULTS_DIR / "cv_results.csv",
439         RESULTS_DIR / "best_params.json",
440         RESULTS_DIR / "feature_importance.csv",
441         RESULTS_DIR / "sensor_ablation_results.csv",
442         RESULTS_DIR / "sensor_summary_statistics.csv",

```

```

443     FIGURES_DIR / "class_distribution.png",
444     FIGURES_DIR / "pca_sensor_space.png",
445     FIGURES_DIR / "model_comparison_f1.png",
446     FIGURES_DIR / "model_comparison_accuracy.png",
447     FIGURES_DIR / "model_comparison_f1_all.png",
448     FIGURES_DIR / "model_comparison_accuracy_all.png",
449     FIGURES_DIR / "confusion_matrix_best_model.png",
450     FIGURES_DIR / "confusion_matrix_best_overall_model.png",
451     FIGURES_DIR / "cv_score_comparison.png",
452     FIGURES_DIR / "sensor_ablation_comparison.png",
453     FIGURES_DIR / "sensor_coefficients_logistic_regression.png",
454     FIGURES_DIR / "sensor_coefficients_linear_svm.png",
455 ]
456 for model_slug in model_slugs:
457     required_paths.extend(
458         [
459             CLASSIFICATION_REPORT_DIR / f"{model_slug}.txt",
460             CONFUSION_MATRIX_DIR / f"{model_slug}.csv",
461             TRAINED_MODEL_DIR / f"{model_slug}.joblib",
462             FIGURES_DIR / f"confusion_matrix_{model_slug}.png",
463         ]
464     )
465     return [str(path.relative_to(PROJECT_ROOT)) for path in required_paths if not path.exists()]
466
467
468 def main() -> None:
469     ensure_project_directories()
470     download_all_datasets()
471     datasets = load_all_datasets()
472     primary = datasets["24_sensors"]
473     print_dataset_summary(primary)
474
475     train_idx, test_idx = make_train_test_indices(primary.y)
476     print_split_summary(primary, train_idx, test_idx)
477     write_dataset_verification(primary, train_idx, test_idx)
478     generate_eda_outputs(primary)
479     metrics, main_best_params, confusion_matrices, feature_importance = run_main_experiment(
480         primary, train_idx, test_idx
481     )
482     ablation, ablation_best_params = run_sensor_ablation(datasets, train_idx, test_idx)
483     best_params = {
484         "full_24_sensor_experiment": main_best_params,
485         "sensor_ablation": ablation_best_params,
486     }
487     write_json(RESULTS_DIR / "best_params.json", best_params)
488
489     context = build_context(
490         primary, metrics, best_params, ablation, feature_importance, confusion_matrices
491     )
492
493     missing_outputs = verify_required_outputs()
494     if missing_outputs:
495         raise RuntimeError(f"Missing required outputs: {missing_outputs}")
496
497     best_row = context["best_row"]
498     print("\nFinal summary")
499     print("=====")
500     print(f"Dataset shape: {primary.data.shape[0]} samples x {primary.data.shape[1]} columns")
501     print("Models trained: Dummy Majority, Logistic Regression, Linear SVM, RBF SVM, MLP Neural Network")
502     print(f"Best model: {context['best_model_name']}")
503     print(f"Best test accuracy: {format_metric(float(best_row['test_accuracy']))}")
504     print(f"Best test macro F1: {format_metric(float(best_row['test_f1_macro']))}")
505     print(f"Best hyperparameters: {json.dumps(context['best_params'], sort_keys=True)}")
506     print(f"Most confused command pair: {context['confusion']['description']}")
507     print(f"Best class-report model: {context['class_best_model_name']}")
508     print(f"Class-report test accuracy: {format_metric(float(context['class_best_row']['test_accuracy']))}")
509     print(f"Class-report test macro F1: {format_metric(float(context['class_best_row']['test_f1_macro']))}")
510     print(f"Class-report confusion pair: {context['class_confusion']['description']}")
511     print(
512         "Nonlinear beat linear: "
513         f"{'yes' if context['nonlinear_beats_linear'] else 'no'} "
514         f"({context['best_nonlinear_model']} {format_metric(context['best_nonlinear_f1'])} vs. "
515         f"{context['best_linear_model']} {format_metric(context['best_linear_f1'])})"
516     )
517     print(
518         "Sensor reduction hurt performance: "
519         f"{'yes' if context['sensor_reduction_hurt'] else 'no'} "
520         f"(24-sensor best {format_metric(context['full_ablation_f1'])}, "
521         f"best reduced {format_metric(context['reduced_best_f1'])})"
522     )
523     if not feature_importance.empty:
524         top = (
525             feature_importance.sort_values("mean_abs_coefficient", ascending=False)
526             .head(5)[["model", "feature", "sensor_region", "mean_abs_coefficient"]]
527             .to_dict(orient="records")
528         )
529         print(f"Most important sensors by coefficient magnitude: {top}")
530     print("Files created/updated: results CSV/JSON/TXT files, trained models, and figures.")
531     print("Reports are static files in report/ and are not generated by main.py")

```



```
532
533
534 if __name__ == "__main__":
535     main()
```

src/config.py

```
1 from pathlib import Path
2
3
4 PROJECT_ROOT = Path(__file__).resolve().parents[1]
5 DATA_DIR = PROJECT_ROOT / "data"
6 RAW_DATA_DIR = DATA_DIR / "raw"
7 PROCESSED_DATA_DIR = DATA_DIR / "processed"
8 FIGURES_DIR = PROJECT_ROOT / "figures"
9 RESULTS_DIR = PROJECT_ROOT / "results"
10 CLASSIFICATION_REPORT_DIR = RESULTS_DIR / "classification_reports"
11 CONFUSION_MATRIX_DIR = RESULTS_DIR / "confusion_matrices"
12 TRAINED_MODEL_DIR = RESULTS_DIR / "trained_models"
13
14 RANDOM_STATE = 42
15 TEST_SIZE = 0.20
16 N_SPLITS = 5
17 PRIMARY_SCORING = "f1_macro"
18
19 TARGET_COLUMN = "command"
20 COMMAND_ORDER = [
21     "Move-Forward",
22     "Slight-Right-Turn",
23     "Sharp-Right-Turn",
24     "Slight-Left-Turn",
25 ]
26
27 UCI_BASE_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/00194"
28 DATASET_URLS = {
29     "24_sensors": f"{UCI_BASE_URL}/sensor_readings_24.data",
30     "4_sensors": f"{UCI_BASE_URL}/sensor_readings_4.data",
31     "2_sensors": f"{UCI_BASE_URL}/sensor_readings_2.data",
32 }
33
34 DATASET_FILES = {
35     "24_sensors": "sensor_readings_24.data",
36     "4_sensors": "sensor_readings_4.data",
37     "2_sensors": "sensor_readings_2.data",
38 }
39
40 DATASET_DISPLAY_NAMES = {
41     "24_sensors": "24 ultrasonic sensors",
42     "4_sensors": "Official 4-sensor version",
43     "2_sensors": "Official 2-sensor version",
44 }
45
46 FEATURE_NAMES = {
47     "24_sensors": [f"US_{idx}" for idx in range(1, 25)],
48     "4_sensors": ["US_front", "US_left", "US_right", "US_back"],
49     "2_sensors": ["US_front", "US_left"],
50 }
51
52 SENSOR_ANGLES_DEGREES = {
53     f"US_{idx}": angle
54     for idx, angle in enumerate(
55         [180, -165, -150, -135, -120, -105, -90, -75, -60, -45, -30, -15,
56          0, 15, 30, 45, 60, 75, 90, 105, 120, 135, 150, 165],
57         start=1,
58     )
59 }
60
61
62 def sensor_region(feature_name: str) -> str:
63     """Return an approximate robot-relative region for a 24-sensor feature."""
64     angle = SENSOR_ANGLES_DEGREES.get(feature_name)
65     if angle is None:
66         return "unknown"
67     if abs(angle) >= 150:
68         return "front"
69     if abs(angle) <= 30:
70         return "back"
71     if angle < 0:
72         return "right"
73     return "left"
```

src/download_data.py

```
1 from __future__ import annotations
2
3 import urllib.error
4 import urllib.request
5 from pathlib import Path
6
7 from .config import DATASET_FILES, DATASET_URLS, RAW_DATA_DIR
8
9
10 def download_file(url: str, destination: Path, timeout_seconds: int = 60) -> Path:
11     """Download one UCI data file if it does not already exist locally."""
12     destination.parent.mkdir(parents=True, exist_ok=True)
13     if destination.exists() and destination.stat().st_size > 0:
14         print(f"Found existing raw file: {destination.name}")
15         return destination
16
17     print(f"Downloading {url}")
18     try:
19         with urllib.request.urlopen(url, timeout=timeout_seconds) as response:
20             destination.write_bytes(response.read())
21     except (urllib.error.URLError, TimeoutError) as exc:
22         raise FileNotFoundError(
23             "Could not download the UCI Wall-Following Robot Navigation data. "
24             f"Please download {url} manually and place it at {destination}."
25         ) from exc
26
27     if destination.stat().st_size == 0:
28         destination.unlink(missing_ok=True)
29         raise FileNotFoundError(
30             f"The downloaded file was empty. Please place the raw file at {destination}."
31         )
32     return destination
33
34
35 def download_all_datasets(raw_dir: Path = RAW_DATA_DIR) -> dict[str, Path]:
36     """Download all official UCI sensor versions used by the project."""
37     downloaded_paths = {}
38     for dataset_key, url in DATASET_URLS.items():
39         path = raw_dir / DATASET_FILES[dataset_key]
40         downloaded_paths[dataset_key] = download_file(url, path)
41     return downloaded_paths
```

src/load_data.py

```
1 from __future__ import annotations
2
3 from dataclasses import dataclass
4 from pathlib import Path
5
6 import pandas as pd
7
8 from .config import (
9     DATASET_DISPLAY_NAMES,
10    DATASET_FILES,
11    FEATURE_NAMES,
12    PROCESSED_DATA_DIR,
13    RAW_DATA_DIR,
14    TARGET_COLUMN,
15 )
16
17
18 @dataclass(frozen=True)
19 class RobotDataset:
20     key: str
21     display_name: str
22     data: pd.DataFrame
23     feature_names: list[str]
24     target_name: str
25     source_note: str
26
27     @property
28     def X(self) -> pd.DataFrame:
29         return self.data[self.feature_names]
30
31     @property
32     def y(self) -> pd.Series:
33         return self.data[self.target_name]
34
35
36 def _read_sensor_file(path: Path, feature_names: list[str]) -> pd.DataFrame:
37     expected_columns = feature_names + [TARGET_COLUMN]
38     if not path.exists():
39         raise FileNotFoundError(
40             f"Missing raw dataset file: {path}. Download it from the UCI archive "
41             f"and place it in {path.parent}."
42         )
43     data = pd.read_csv(path, header=None, names=expected_columns)
44     for column in feature_names:
45         data[column] = pd.to_numeric(data[column], errors="coerce")
46     data[TARGET_COLUMN] = data[TARGET_COLUMN].astype(str).str.strip()
47     return data
48
49
50 def load_dataset(dataset_key: str, raw_dir: Path = RAW_DATA_DIR) -> RobotDataset:
51     """Load one official UCI dataset version and save a processed CSV copy."""
52     feature_names = FEATURE_NAMES[dataset_key]
53     raw_path = raw_dir / DATASET_FILES[dataset_key]
54     data = _read_sensor_file(raw_path, feature_names)
55     processed_path = PROCESSED_DATA_DIR / f"{dataset_key}.csv"
56     processed_path.parent.mkdir(parents=True, exist_ok=True)
57     data.to_csv(processed_path, index=False)
58     return RobotDataset(
59         key=dataset_key,
60         display_name=DATASET_DISPLAY_NAMES[dataset_key],
61         data=data,
62         feature_names=feature_names,
63         target_name=TARGET_COLUMN,
64         source_note="Official UCI file",
65     )
66
67
68 def load_all_datasets(raw_dir: Path = RAW_DATA_DIR) -> dict[str, RobotDataset]:
69     """Load the 24-, 4-, and 2-sensor UCI files."""
70     return {
71         dataset_key: load_dataset(dataset_key, raw_dir=raw_dir)
72         for dataset_key in ("24_sensors", "4_sensors", "2_sensors")
73     }
74
75
76 def summarize_missing_values(dataset: RobotDataset) -> pd.Series:
77     return dataset.data.isna().sum()
```

src/train_models.py

```
1 from __future__ import annotations
2
3 import warnings
4 from dataclasses import dataclass
5
6 import pandas as pd
7 from sklearn.dummy import DummyClassifier
8 from sklearn.exceptions import ConvergenceWarning
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.model_selection import GridSearchCV, StratifiedKFold
11 from sklearn.neural_network import MLPClassifier
12 from sklearn.pipeline import Pipeline
13 from sklearn.preprocessing import StandardScaler
14 from sklearn.svm import SVC
15
16 from .config import N_SPLITS, PRIMARY_SCORING, RANDOM_STATE
17
18
19 @dataclass(frozen=True)
20 class ModelSpec:
21     name: str
22     pipeline: Pipeline
23     param_grid: dict[str, list]
24
25
26 def get_main_model_specs(random_state: int = RANDOM_STATE) -> list[ModelSpec]:
27     return [
28         ModelSpec(
29             name="Dummy Majority",
30             pipeline=Pipeline(
31                 [("model", DummyClassifier(strategy="most_frequent", random_state=random_state))]
32             ),
33             param_grid={"model__strategy": ["most_frequent"]},
34         ),
35         ModelSpec(
36             name="Logistic Regression",
37             pipeline=Pipeline(
38                 [
39                     ("scaler", StandardScaler()),
40                     (
41                         "model",
42                         LogisticRegression(
43                             solver="lbfgs",
44                             penalty="l2",
45                             max_iter=5000,
46                             random_state=random_state,
47                         ),
48                     ),
49                 ]
50             ),
51             param_grid={
52                 "model__C": [0.01, 0.1, 1, 10, 100],
53                 "model__penalty": ["l2"],
54             },
55         ),
56         ModelSpec(
57             name="Linear SVM",
58             pipeline=Pipeline(
59                 [
60                     ("scaler", StandardScaler()),
61                     ("model", SVC(kernel="linear", random_state=random_state)),
62                 ]
63             ),
64             param_grid={"model__C": [0.01, 0.1, 1, 10, 100]},
65         ),
66         ModelSpec(
67             name="RBF SVM",
68             pipeline=Pipeline(
69                 [
70                     ("scaler", StandardScaler()),
71                     ("model", SVC(kernel="rbf", random_state=random_state)),
72                 ]
73             ),
74             param_grid={
75                 "model__C": [0.1, 1, 10, 100],
76                 "model__gamma": ["scale", 0.001, 0.01, 0.1, 1],
77             },
78         ),
79         ModelSpec(
80             name="MLP Neural Network",
81             pipeline=Pipeline(
82                 [
83                     ("scaler", StandardScaler()),
84                     (
85                         "model",
86                         MLPClassifier(
```

```

87         max_iter=1000,
88         n_iter_no_change=25,
89         random_state=random_state,
90     ),
91 ],
92 ],
93 ),
94 param_grid={
95     "model__hidden_layer_sizes": [(16,), (32,), (64,), (32, 16)],
96     "model__alpha": [0.0001, 0.001, 0.01],
97     "model__activation": ["relu", "tanh"],
98 },
99 ),
100 ]
101
102
103 def get_ablation_model_specs(random_state: int = RANDOM_STATE) -> list[ModelSpec]:
104     return [
105         ModelSpec(
106             name="Logistic Regression",
107             pipeline=Pipeline(
108                 [
109                     ("scaler", StandardScaler()),
110                     (
111                         "model",
112                         LogisticRegression(
113                             solver="lbfgs",
114                             penalty="l2",
115                             max_iter=5000,
116                             random_state=random_state,
117                         ),
118                     ),
119                 ]
120             ),
121             param_grid={"model__C": [0.1, 1, 10], "model__penalty": ["l2"]},
122         ),
123         ModelSpec(
124             name="RBF SVM",
125             pipeline=Pipeline(
126                 [
127                     ("scaler", StandardScaler()),
128                     ("model", SVC(kernel="rbf", random_state=random_state)),
129                 ]
130             ),
131             param_grid={
132                 "model__C": [1, 10, 100],
133                 "model__gamma": ["scale", 0.01, 0.1],
134             },
135         ),
136         ModelSpec(
137             name="MLP Neural Network",
138             pipeline=Pipeline(
139                 [
140                     ("scaler", StandardScaler()),
141                     (
142                         "model",
143                         MLPClassifier(
144                             max_iter=1000,
145                             n_iter_no_change=25,
146                             random_state=random_state,
147                         ),
148                     ),
149                 ]
150             ),
151             param_grid={
152                 "model__hidden_layer_sizes": [(32,), (64,)],
153                 "model__alpha": [0.0001, 0.001],
154                 "model__activation": ["relu"],
155             },
156         ),
157     ]
158
159
160 def fit_grid_search(spec: ModelSpec, X_train, y_train) -> GridSearchCV:
161     cv = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
162     search = GridSearchCV(
163         estimator=spec.pipeline,
164         param_grid=spec.param_grid,
165         scoring=PRIMARY_SCORING,
166         cv=cv,
167         n_jobs=-1,
168         refit=True,
169         return_train_score=True,
170     )
171     with warnings.catch_warnings():
172         warnings.filterwarnings("ignore", category=FutureWarning)
173         warnings.filterwarnings("ignore", category=ConvergenceWarning)
174         search.fit(X_train, y_train)
175     return search

```

```

176
177
178 def cv_results_to_frame(model_name: str, search: GridSearchCV, dataset_key: str) -> pd.DataFrame:
179     frame = pd.DataFrame(search.cv_results_)
180     keep_columns = [
181         "mean_test_score",
182         "std_test_score",
183         "mean_train_score",
184         "std_train_score",
185         "rank_test_score",
186         "params",
187     ]
188     frame = frame[keep_columns].copy()
189     frame.insert(0, "dataset", dataset_key)
190     frame.insert(1, "model", model_name)
191     frame["params"] = frame["params"].astype(str)
192     return frame.sort_values(["dataset", "model", "rank_test_score"])

```

src/evaluate_models.py

```
1 from __future__ import annotations
2
3 import pandas as pd
4 from sklearn.metrics import (
5     accuracy_score,
6     classification_report,
7     confusion_matrix,
8     precision_score,
9     recall_score,
10    f1_score,
11 )
12
13 from .config import COMMAND_ORDER, sensor_region
14
15
16 def compute_metric_block(y_true, y_pred, prefix: str) -> dict[str, float]:
17     return {
18         f"{prefix}_accuracy": accuracy_score(y_true, y_pred),
19         f"{prefix}_precision_macro": precision_score(
20             y_true, y_pred, average="macro", zero_division=0
21         ),
22         f"{prefix}_recall_macro": recall_score(
23             y_true, y_pred, average="macro", zero_division=0
24         ),
25         f"{prefix}_f1_macro": f1_score(y_true, y_pred, average="macro", zero_division=0),
26     }
27
28
29 def evaluate_estimator(
30     model_name: str,
31     estimator,
32     X_train,
33     y_train,
34     X_test,
35     y_test,
36     labels: list[str] = COMMAND_ORDER,
37 ) -> tuple[dict[str, float], str, pd.DataFrame]:
38     train_pred = estimator.predict(X_train)
39     test_pred = estimator.predict(X_test)
40     metrics = {"model": model_name}
41     metrics.update(compute_metric_block(y_train, train_pred, "train"))
42     metrics.update(compute_metric_block(y_test, test_pred, "test"))
43
44     train_report = classification_report(
45         y_train, train_pred, labels=labels, zero_division=0
46     )
47     test_report = classification_report(y_test, test_pred, labels=labels, zero_division=0)
48     report_text = (
49         f"Model: {model_name}\n\n"
50         "Training classification report\n"
51         "=====\n"
52         f"{train_report}\n\n"
53         "Held-out test classification report\n"
54         "=====\n"
55         f"{test_report}\n"
56     )
57
58     cm = confusion_matrix(y_test, test_pred, labels=labels)
59     cm_frame = pd.DataFrame(cm, index=labels, columns=labels)
60     return metrics, report_text, cm_frame
61
62
63 def most_confused_pair(cm_frame: pd.DataFrame) -> dict[str, object]:
64     off_diagonal = cm_frame.copy()
65     for label in cm_frame.index:
66         off_diagonal.loc[label, label] = 0
67     max_count = int(off_diagonal.to_numpy().max())
68     if max_count == 0:
69         return {
70             "true_label": "None",
71             "predicted_label": "None",
72             "count": 0,
73             "description": "No off-diagonal confusions in the held-out test set.",
74         }
75     true_label, predicted_label = off_diagonal.stack().idxmax()
76     return {
77         "true_label": true_label,
78         "predicted_label": predicted_label,
79         "count": max_count,
80         "description": f"{true_label} predicted as {predicted_label} ({max_count} test samples)",
81     }
82
83
84 def extract_linear_feature_importance(
85     model_name: str,
86     estimator,
```



```

87     feature_names: list[str],
88 ) -> pd.DataFrame:
89     model = estimator.named_steps.get("model")
90     if model is None or not hasattr(model, "coef_"):
91         return pd.DataFrame()
92
93     coef = model.coef_
94     classes = list(getattr(model, "classes_", []))
95     rows = []
96     for feature_idx, feature_name in enumerate(feature_names):
97         class_coefficients = {
98             f"coef_{class_label}": coef[class_idx, feature_idx]
99             for class_idx, class_label in enumerate(classes)
100         }
101         abs_values = [abs(value) for value in class_coefficients.values()]
102         rows.append(
103             {
104                 "model": model_name,
105                 "feature": feature_name,
106                 "sensor_region": sensor_region(feature_name),
107                 "mean_abs_coefficient": sum(abs_values) / len(abs_values),
108                 "max_abs_coefficient": max(abs_values),
109                 **class_coefficients,
110             }
111         )
112
113     return pd.DataFrame(rows).sort_values(
114         ["model", "mean_abs_coefficient"], ascending=[True, False]
115     )

```

src/plot_results.py

```
1 from __future__ import annotations
2
3 import matplotlib
4
5 matplotlib.use("Agg")
6
7 import matplotlib.pyplot as plt
8 import pandas as pd
9 import seaborn as sns
10 from sklearn.decomposition import PCA
11 from sklearn.preprocessing import StandardScaler
12
13 from .config import COMMAND_ORDER
14
15
16 PLOT_STYLE = "whitegrid"
17 FIGURE_DPI = 180
18
19
20 def _set_style() -> None:
21     sns.set_theme(style=PLOT_STYLE, context="notebook", font_scale=1.05)
22
23
24 def save_class_distribution(y: pd.Series, output_path) -> None:
25     _set_style()
26     counts = y.value_counts().reindex(COMMAND_ORDER)
27     fig, ax = plt.subplots(figsize=(9, 5))
28     sns.barplot(
29         x=counts.index,
30         y=counts.values,
31         hue=counts.index,
32         legend=False,
33         ax=ax,
34         palette="Set2",
35     )
36     ax.set_title("Class Distribution of Robot Navigation Commands")
37     ax.set_xlabel("Movement command")
38     ax.set_ylabel("Number of samples")
39     ax.tick_params(axis="x", rotation=20)
40     for idx, value in enumerate(counts.values):
41         ax.text(idx, value, str(int(value)), ha="center", va="bottom", fontsize=10)
42     fig.tight_layout()
43     fig.savefig(output_path, dpi=FIGURE_DPI)
44     plt.close(fig)
45
46
47 def save_pca_sensor_space(X: pd.DataFrame, y: pd.Series, output_path) -> None:
48     _set_style()
49     scaled = StandardScaler().fit_transform(X)
50     pca = PCA(n_components=2, random_state=42)
51     projected = pca.fit_transform(scaled)
52     frame = pd.DataFrame(
53         {
54             "PC1": projected[:, 0],
55             "PC2": projected[:, 1],
56             "command": y.to_numpy(),
57         }
58     )
59     fig, ax = plt.subplots(figsize=(8, 6))
60     sns.scatterplot(
61         data=frame,
62         x="PC1",
63         y="PC2",
64         hue="command",
65         hue_order=COMMAND_ORDER,
66         s=18,
67         alpha=0.65,
68         linewidth=0,
69         ax=ax,
70         palette="Set2",
71     )
72     explained = pca.explained_variance_ratio_ * 100
73     ax.set_title("PCA Projection of Ultrasonic Sensor Readings")
74     ax.set_xlabel(f"PC1 ({explained[0]:.1f}% variance)")
75     ax.set_ylabel(f"PC2 ({explained[1]:.1f}% variance)")
76     ax.legend(title="Command", bbox_to_anchor=(1.02, 1), loc="upper left")
77     fig.tight_layout()
78     fig.savefig(output_path, dpi=FIGURE_DPI)
79     plt.close(fig)
80
81
82 def save_sensor_correlation_heatmap(X: pd.DataFrame, output_path) -> None:
83     _set_style()
84     fig, ax = plt.subplots(figsize=(10, 8))
85     sns.heatmap(X.corr(), cmap="vlag", center=0, square=True, cbar_kws={"shrink": 0.8}, ax=ax)
86     ax.set_title("Correlation Heatmap of 24 Ultrasonic Sensors")
```

```

87     ax.set_xlabel("Sensor")
88     ax.set_ylabel("Sensor")
89     fig.tight_layout()
90     fig.savefig(output_path, dpi=FIGURE_DPI)
91     plt.close(fig)
92
93
94 def save_metric_comparison(metrics: pd.DataFrame, metric: str, output_path, title: str, y_label: str) -> None:
95     _set_style()
96     display = metrics.melt(
97         id_vars=["model"],
98         value_vars=[f"train_{metric}", f"test_{metric}"],
99         var_name="split",
100        value_name="score",
101    )
102    display["split"] = display["split"].map(
103        {f"train_{metric}": "Training", f"test_{metric}": "Held-out test"}
104    )
105    fig, ax = plt.subplots(figsize=(10, 5.5))
106    sns.barplot(data=display, x="model", y="score", hue="split", ax=ax, palette="Set2")
107    ax.set_title(title)
108    ax.set_xlabel("Model")
109    ax.set_ylabel(y_label)
110    ax.set_ylim(0, min(1.05, max(0.1, display["score"].max() + 0.08)))
111    ax.tick_params(axis="x", rotation=20)
112    ax.legend(title="Split")
113    fig.tight_layout()
114    fig.savefig(output_path, dpi=FIGURE_DPI)
115    plt.close(fig)
116
117
118 def save_cv_score_comparison(metrics: pd.DataFrame, output_path) -> None:
119     _set_style()
120     fig, ax = plt.subplots(figsize=(9, 5))
121     sns.barplot(
122         data=metrics,
123         x="model",
124         y="cv_best_macro_f1",
125         hue="model",
126         legend=False,
127         ax=ax,
128         palette="Set2",
129     )
130     ax.set_title("Best 5-Fold Cross-Validation Macro F1 by Model")
131     ax.set_xlabel("Model")
132     ax.set_ylabel("Best CV macro F1")
133     ax.set_ylim(0, min(1.05, max(0.1, metrics["cv_best_macro_f1"].max() + 0.08)))
134     ax.tick_params(axis="x", rotation=20)
135     fig.tight_layout()
136     fig.savefig(output_path, dpi=FIGURE_DPI)
137     plt.close(fig)
138
139
140 def save_confusion_matrix(cm_frame: pd.DataFrame, output_path, title: str) -> None:
141     _set_style()
142     fig, ax = plt.subplots(figsize=(7, 6))
143     sns.heatmap(
144         cm_frame,
145         annot=True,
146         fmt="d",
147         cmap="Blues",
148         cbar=False,
149         square=True,
150         ax=ax,
151     )
152     ax.set_title(title)
153     ax.set_xlabel("Predicted command")
154     ax.set_ylabel("True command")
155     ax.tick_params(axis="x", rotation=25)
156     ax.tick_params(axis="y", rotation=0)
157     fig.tight_layout()
158     fig.savefig(output_path, dpi=FIGURE_DPI)
159     plt.close(fig)
160
161
162 def save_sensor_ablation_comparison(ablation: pd.DataFrame, output_path) -> None:
163     _set_style()
164     fig, ax = plt.subplots(figsize=(10, 5.5))
165     sns.barplot(
166         data=ablation,
167         x="feature_set",
168         y="test_f1_macro",
169         hue="model",
170         ax=ax,
171         palette="Set2",
172     )
173     ax.set_title("Sensor Ablation Study: Test Macro F1")
174     ax.set_xlabel("Sensor feature set")
175     ax.set_ylabel("Held-out test macro F1")

```

```

176     ax.set_ylim(0, min(1.05, max(0.1, ablation["test_f1_macro"].max() + 0.08)))
177     ax.legend(title="Model", bbox_to_anchor=(1.02, 1), loc="upper left")
178     fig.tight_layout()
179     fig.savefig(output_path, dpi=FIGURE_DPI)
180     plt.close(fig)
181
182
183 def save_coefficient_plot(feature_importance: pd.DataFrame, model_name: str, output_path) -> None:
184     _set_style()
185     data = feature_importance[feature_importance["model"] == model_name].copy()
186     data = data.sort_values("mean_abs_coefficient", ascending=False)
187     fig, ax = plt.subplots(figsize=(11, 5.5))
188     sns.barplot(
189         data=data,
190         x="feature",
191         y="mean_abs_coefficient",
192         hue="sensor_region",
193         dodge=False,
194         ax=ax,
195         palette="Set2",
196     )
197     ax.set_title(f"Sensor Coefficient Magnitudes: {model_name}")
198     ax.set_xlabel("Ultrasonic sensor")
199     ax.set_ylabel("Mean absolute coefficient across classes")
200     ax.tick_params(axis="x", rotation=45)
201     ax.legend(title="Approx. region", bbox_to_anchor=(1.02, 1), loc="upper left")
202     fig.tight_layout()
203     fig.savefig(output_path, dpi=FIGURE_DPI)
204     plt.close(fig)

```

src/utls.py

```
1 from __future__ import annotations
2
3 import json
4 import re
5 from pathlib import Path
6
7 from .config import (
8     CLASSIFICATION_REPORT_DIR,
9     CONFUSION_MATRIX_DIR,
10    DATA_DIR,
11    FIGURES_DIR,
12    PROCESSED_DATA_DIR,
13    RAW_DATA_DIR,
14    RESULTS_DIR,
15    TRAINED_MODEL_DIR,
16 )
17
18
19 def ensure_project_directories() -> None:
20     for path in [
21         DATA_DIR,
22         RAW_DATA_DIR,
23         PROCESSED_DATA_DIR,
24         FIGURES_DIR,
25         RESULTS_DIR,
26         CLASSIFICATION_REPORT_DIR,
27         CONFUSION_MATRIX_DIR,
28         TRAINED_MODEL_DIR,
29     ]:
30         path.mkdir(parents=True, exist_ok=True)
31
32
33 def slugify(value: str) -> str:
34     value = value.lower().replace("+", "plus")
35     value = re.sub(r"^[^a-z0-9]+", "_", value).strip("_")
36     return value
37
38
39 def write_json(path: Path, payload: object) -> None:
40     path.parent.mkdir(parents=True, exist_ok=True)
41     path.write_text(json.dumps(payload, indent=2, sort_keys=True), encoding="utf-8")
42
43
44 def format_metric(value: float) -> str:
45     return f"{value:.3f}"
46
47
48 def write_text(path: Path, text: str) -> None:
49     path.parent.mkdir(parents=True, exist_ok=True)
50     path.write_text(text.strip() + "\n", encoding="utf-8")
```